Jobs and SQL warehouse troubleshooting

Jobs & SQL warehouse troubleshooting (deployment)

Use this checklist when **Databricks Apps**, **scheduled jobs**, or **notebooks** stop loading data, fail with warehouse errors, or show **upstream failed** with no obvious root cause.

---

1. SQL warehouse: what to verify

1.1 Warehouse exists in *this* workspace

Warehouse IDs are **per workspace**. An id copied from another workspace or an old deployment will fail with **warehouse not found**.

```
databricks warehouses list -p <profile> -o json
```

Confirm the id you use in **Apps**, **`app.yml`**, and **jobs** appears in this list.

1.2 Three places that must agree (common drift)

Location	What to check
Databricks App → Resources	SQL warehouse id bound to the app (Apps UI / API `resources`).
`app.yml` → `DATABRICKS_WAREHOUSE_ID`	FastAPI / backend reads this (or `AUTO` + code resolution). Must not point at a deleted id.
Job / notebook widgets	`warehouse_id` parameter or `AUTO` + resolver in notebook source.

If the App resource says **FINS-Apps** but **`app.yml`** still has an old hex id, the UI can look broken while the platform attachment looks fine.

1.3 App service principal vs human users

Apps run as an **app service principal**. Grants to **"all workspace users"** do **not** automatically include that SP. For SQL execution, the SP needs **`CAN_USE`** on the warehouse you actually use.

1.4 Stopped vs broken

A warehouse in **STOPPED** state can **auto-resume** on query. **Not found** / **permission denied** is different from **stopped**.

1.5 Optional: who deleted or replaced a warehouse

Audit (if enabled): query **`system.access.audit`** for actions such as **`deleteWarehouse`**, **`createWarehouse`**, **`stopWarehouse`**. See [Audit log system table reference].

Operational history (not deletion): **`system.compute.warehouse_events`**.

---

2. Jobs: what to verify

2.1 Task dependencies and `run_if`

When task **B** depends on task **A**:

`run_if` on B	If A fails	B runs?	How it looks in the UI
ALL_SUCCESS (default)	Failed	No	B is often **Upstream failed** — the *real* error is on **A**.
ALL_DONE	Failed	Yes	B can **succeed** while A **failed** — you must **inspect A** or you miss the AlphaVantage (or other) failure.

Keep the dependency (edge in the DAG) when you need **ordering** (e.g. wait until A finishes). Choose **`run_if`** based on whether B must still run when A fails.

For **deposit PPNR** daily refresh, if **`full_refresh`** should run even when **`alpha_vantage_refresh`** fails, use **`depends_on`** + **`run_if: ALL_DONE`** on **`full_refresh`**. If **`full_refresh`** must not run without AlphaVantage, use default **ALL_SUCCESS** and rely on the upstream-failure drill-down below.

2.2 Warehouse inside the job

Notebook tasks pass `warehouse_id` (or **`AUTO`**). Errors like **`The warehouse <id> was not found`** come from **SQL statement execution** in the notebook, not from AlphaVantage HTTP success alone.

---

3. Surfacing the *real* reason when something "upstream" failed

`UPSTREAM_FAILED` is not the root cause. It only means a dependency did not satisfy **`run_if`**. The root cause is always in a **specific task run**.

3.1 In the Databricks UI (fastest)

1. Open **Workflows** → the job → **Runs** → select the **parent run**.
2. In the **DAG**, find the task with **Failed** (not merely "skipped").

3. Open that **task run** → **Run output** / **Driver logs** / **notebook output**.
4. Read **`state_message`** on the task (often summarizes the failure).

Order: **failed upstream task first**, then downstream.

3.2 CLI: parent run and per-task runs

```
# Parent run overview (all tasks, states, messages)
databricks jobs get-run <parent_run_id> -p <profile> -o json

# Output for a single task run (when the job has multiple tasks)
databricks jobs get-run-output <task_run_id> -p <profile>
```

From the parent JSON, locate `tasks[]` where `state.result_state` is **`FAILED`** (or **`CANCELED`**). Use that task's `run_id` as **`task_run_id`** for **`get-run-output`**.

3.3 Make failures visible when downstream still runs (`ALL_DONE`)

If downstream tasks use **`ALL_DONE`**, the **parent run** can look "green enough" while an upstream task **failed**. Mitigations:

- **Discipline:** On every run, scan the DAG for **any** red/failed task, not only the last task.
- **Job email notifications:** Configure **on failure** (and optionally **on duration threshold**) so owners get the **failed task** context. See job **Notifications** in the UI.
- **Webhooks / monitoring:** Send job run events to Slack or an observability tool and include **task_key** + **result_state** from the Jobs API.
- **Dedicated "health" task (optional):** Add a small downstream task with **`run_if: AT_LEAST_ONE_FAILED`** (or use an **If/else** condition task) that only runs when an upstream fails and writes a row to a **Delta audit table** or sends a notification — so the failure is **surfaced explicitly** even when refresh tasks still complete.

3.4 Audit trail

`system.access.audit` can help correlate **who** ran or changed jobs; **task-level** errors remain in **job run** output and **`get-run`**.

---

4. Apps + data freshness

- The **App** may still show **old** rows in Delta if **ingestion jobs** fail; the UI does not prove tonight's job succeeded.
- Correlate **job last success time** with **table `updated_at`** or **max partition** if you need freshness checks.

---

5. Quick reference commands

```
# Warehouses
databricks warehouses list -p <profile> -o json

# App (your app only — adjust name)
databricks apps get bank-treasury-deposit-ppnr -p <profile> -o json

# Job settings (parameters, depends_on, run_if)
databricks jobs get <job_id> -p <profile> -o json

# Recent runs
databricks jobs list-runs --job-id <job_id> -p <profile> --limit 10 -o json
```

Related docs

- Configure task dependencies (`run_if`)
- Configure permissions for a Databricks app
- Configure authorization in a Databricks app